

2. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

Gemini 3.6 Flash & Claude Sonnet 5

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both GenAI programs through their websites (claude.ai & gemini.google.com respectively)

c. Prompt used. (You must use the same prompt for both GenAIs.)

Use two GenAI's to generate C code to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

- The program will prioritize emails for a busy company CEO.
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules.

• Test File format:

Command Description

EMAIL <sender category>,
<subject line>,
<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

d. Generated code from both GenAIs.

GEMINI'S GENERATION:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct
{
    char sender[32];
    char subject[256];
    char date[16]; // MM-DD-YYYY
} Email;

typedef struct
{
    Email *data;
    int capacity;
    int size;
} MaxHeap;

// Helper: Convert sender category to numerical rank (Higher = Greater Priority)
int get_sender_rank(const char *sender)
{
    if (strcmp(sender, "Boss") == 0)
        return 5;
    if (strcmp(sender, "Subordinate") == 0)
        return 4;
    if (strcmp(sender, "Peer") == 0)
        return 3;
    if (strcmp(sender, "ImportantPerson") == 0)
        return 2;
    if (strcmp(sender, "OtherPerson") == 0)
        return 1;
    return 0;
}

// Helper: Convert MM-DD-YYYY string to YYYYMMDD integer for direct comparison
int get_date_value(const char *date)
{
    int m, d, y;
    sscanf(date, "%d-%d-%d", &m, &d, &y);
    return y * 10000 + m * 100 + d;
}

```

```

// Returns >0 if e1 has higher priority than e2
int compare_emails(Email e1, Email e2)
{
    int rank1 = get_sender_rank(e1.sender);
    int rank2 = get_sender_rank(e2.sender);

    if (rank1 != rank2)
    {
        return rank1 - rank2;
    }

    // Tie-breaker: Newer date gets higher priority
    return get_date_value(e1.date) - get_date_value(e2.date);
}

// MaxHeap Initialization
MaxHeap *create_heap(int capacity)
{
    MaxHeap *heap = (MaxHeap *)malloc(sizeof(MaxHeap));
    heap->capacity = capacity;
    heap->size = 0;
    heap->data = (Email *)malloc(sizeof(Email) * capacity);
    return heap;
}

void swap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapify_up(MaxHeap *heap, int index)
{
    while (index > 0)
    {
        int parent = (index - 1) / 2;
        if (compare_emails(heap->data[index], heap->data[parent]) > 0)
        {
            swap(&heap->data[index], &heap->data[parent]);
            index = parent;
        }
    }
}

```

```

        else
        {
            break;
        }
    }
}

void heapify_down(MaxHeap *heap, int index)
{
    while (2 * index + 1 < heap->size)
    {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < heap->size && compare_emails(heap->data[left], heap->data[largest])
> 0)
        {
            largest = left;
        }
        if (right < heap->size && compare_emails(heap->data[right],
heap->data[largest]) > 0)
        {
            largest = right;
        }

        if (largest != index)
        {
            swap(&heap->data[index], &heap->data[largest]);
            index = largest;
        }
        else
        {
            break;
        }
    }
}

void push(MaxHeap *heap, Email email)
{
    if (heap->size == heap->capacity)
    {

```

```

        heap->capacity *= 2;
        heap->data = (Email *)realloc(heap->data, sizeof(Email) * heap->capacity);
    }
    heap->data[heap->size] = email;
    heapify_up(heap, heap->size);
    heap->size++;
}

void pop(MaxHeap *heap)
{
    if (heap->size == 0)
        return;
    heap->data[0] = heap->data[heap->size - 1];
    heap->size--;
    heapify_down(heap, 0);
}

Email *peek(MaxHeap *heap)
{
    if (heap->size == 0)
        return NULL;
    return &heap->data[0];
}

void free_heap(MaxHeap *heap)
{
    free(heap->data);
    free(heap);
}

int main()
{
    MaxHeap *heap = create_heap(10);
    char line[512];

    while (fgets(line, sizeof(line), stdin))
    {
        // Strip trailing newline character
        line[strcspn(line, "\r\n")] = 0;

        if (strncmp(line, "EMAIL ", 6) == 0)
        {

```

```

Email e;
char *payload = line + 6;

// Parse comma-delimited fields
char *sender = strtok(payload, ",");
char *subject = strtok(NULL, ",");
char *date = strtok(NULL, ",");

if (sender && subject && date)
{
    // Trim leading whitespace from tokens if present
    while (*sender == ' ')
        sender++;
    while (*subject == ' ')
        subject++;
    while (*date == ' ')
        date++;

    strcpy(e.sender, sender);
    strcpy(e.subject, subject);
    strcpy(e.date, date);
    push(heap, e);
}
}
else if (strcmp(line, "COUNT") == 0)
{
    printf("There are %d emails to read.\n", heap->size);
}
else if (strcmp(line, "NEXT") == 0)
{
    Email *next_email = peek(heap);
    if (next_email)
    {
        printf("Next email:\nSender: %s\nSubject: %s\nDate: %s\n",
            next_email->sender, next_email->subject, next_email->date);
    }
}
else if (strcmp(line, "READ") == 0)
{
    pop(heap);
}
}

```

```

    free_heap(heap);
    return 0;
}

```

CLAUDE'S GENERATION:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* -----
   Email record
   ----- */

#define MAX_CATEGORY_LEN 32
#define MAX_SUBJECT_LEN 256
#define MAX_DATE_LEN 16
#define MAX_LINE_LEN 512

typedef struct
{
    char category[MAX_CATEGORY_LEN]; /* original category string */
    char subject[MAX_SUBJECT_LEN];
    char date[MAX_DATE_LEN]; /* MM-DD-YYYY as given */
    int categoryRank; /* higher = more important */
    long dateValue; /* YYYYMMDD, higher = newer */
} Email;

/* -----
   MaxHeap (array/list based implementation)
   ----- */

typedef struct
{
    Email *data; /* dynamic array of emails */
    int size; /* current number of emails in the heap */
    int capacity; /* current allocated capacity of the array */
} MaxHeap;

/* ----- Heap helper prototypes ----- */
static void heapInit(MaxHeap *h);
static void heapFree(MaxHeap *h);
static void heapEnsureCapacity(MaxHeap *h);

```

```

static void heapSwap(Email *a, Email *b);
static int emailIsHigherPriority(const Email *a, const Email *b);
static void heapifyUp(MaxHeap *h, int index);
static void heapifyDown(MaxHeap *h, int index);
static void heapInsert(MaxHeap *h, Email e);
static int heapExtractMax(MaxHeap *h, Email *out); /* returns 1 on success, 0 if empty
*/
static int heapPeekMax(const MaxHeap *h, Email *out);

/* ----- Parsing helper prototypes ----- */
static int categoryRank(const char *category);
static long dateToValue(const char *date);
static void trimNewline(char *s);
static void printEmail(const Email *e);

/* =====
MAIN
===== */
int main(void)
{
    MaxHeap heap;
    heapInit(&heap);

    char line[MAX_LINE_LEN];

    while (fgets(line, sizeof(line), stdin) != NULL)
    {
        trimNewline(line);

        /* Skip completely blank lines */
        if (line[0] == '\0')
        {
            continue;
        }

        if (strncmp(line, "EMAIL ", 6) == 0)
        {
            /* Rest of line after "EMAIL " : category,subject,date */
            char *rest = line + 6;

            char category[MAX_CATEGORY_LEN] = {0};
            char subject[MAX_SUBJECT_LEN] = {0};

```

```

char date[MAX_DATE_LEN] = {0};

/* First comma separates category */
char *firstComma = strchr(rest, ',');
if (firstComma == NULL)
{
    /* Malformed line; skip it */
    continue;
}
/* Last comma separates date (subject itself has no commas,
   so the LAST comma in the string is the one before date) */
char *lastComma = strrchr(rest, ',');
if (lastComma == firstComma)
{
    /* Only one comma found -> malformed (need category,subject,date) */
    continue;
}

size_t catLen = (size_t)(firstComma - rest);
if (catLen >= sizeof(category))
    catLen = sizeof(category) - 1;
strncpy(category, rest, catLen);
category[catLen] = '\0';

size_t subjLen = (size_t)(lastComma - (firstComma + 1));
if (subjLen >= sizeof(subject))
    subjLen = sizeof(subject) - 1;
strncpy(subject, firstComma + 1, subjLen);
subject[subjLen] = '\0';

strncpy(date, lastComma + 1, sizeof(date) - 1);
date[sizeof(date) - 1] = '\0';

Email e;
strncpy(e.category, category, sizeof(e.category) - 1);
e.category[sizeof(e.category) - 1] = '\0';
strncpy(e.subject, subject, sizeof(e.subject) - 1);
e.subject[sizeof(e.subject) - 1] = '\0';
strncpy(e.date, date, sizeof(e.date) - 1);
e.date[sizeof(e.date) - 1] = '\0';

e.categoryRank = categoryRank(e.category);

```

```

        e.dateValue = dateToValue(e.date);

        heapInsert(&heap, e);
    }
    else if (strcmp(line, "NEXT") == 0)
    {
        Email e;
        if (heapPeekMax(&heap, &e))
        {
            printf("Next email:\n");
            printEmail(&e);
        }
        else
        {
            printf("No emails to read.\n");
        }
    }
    else if (strcmp(line, "READ") == 0)
    {
        Email e;
        /* Simply remove the top email; no output required.
           If the queue is empty, do nothing (no crash). */
        heapExtractMax(&heap, &e);
    }
    else if (strcmp(line, "COUNT") == 0)
    {
        printf("There are %d emails to read.\n", heap.size);
    }
    /* Any other/unknown line is silently ignored */
}

heapFree(&heap);
return 0;
}

/* =====
MaxHeap implementation (array / list based, built from scratch)
===== */

static void heapInit(MaxHeap *h)
{
    h->capacity = 8;

```

```

    h->size = 0;
    h->data = (Email *)malloc(sizeof(Email) * (size_t)h->capacity);
    if (h->data == NULL)
    {
        fprintf(stderr, "Memory allocation failed.\n");
        exit(1);
    }
}

static void heapFree(MaxHeap *h)
{
    free(h->data);
    h->data = NULL;
    h->size = 0;
    h->capacity = 0;
}

static void heapEnsureCapacity(MaxHeap *h)
{
    if (h->size >= h->capacity)
    {
        h->capacity *= 2;
        Email *newData = (Email *)realloc(h->data, sizeof(Email) *
(size_t)h->capacity);
        if (newData == NULL)
        {
            fprintf(stderr, "Memory allocation failed.\n");
            exit(1);
        }
        h->data = newData;
    }
}

static void heapSwap(Email *a, Email *b)
{
    Email temp = *a;
    *a = *b;
    *b = temp;
}

/* Returns 1 if email 'a' should be considered higher priority (i.e.
should sit closer to the root / be read sooner) than email 'b'.

```

```

Rules:
    1. Higher categoryRank wins.
    2. If categories tie, the newer date (larger dateValue) wins. */
static int emailIsHigherPriority(const Email *a, const Email *b)
{
    if (a->categoryRank != b->categoryRank)
    {
        return a->categoryRank > b->categoryRank;
    }
    return a->dateValue > b->dateValue;
}

/* Bubble the element at 'index' up until heap property holds */
static void heapifyUp(MaxHeap *h, int index)
{
    while (index > 0)
    {
        int parent = (index - 1) / 2;
        if (emailIsHigherPriority(&h->data[index], &h->data[parent]))
        {
            heapSwap(&h->data[index], &h->data[parent]);
            index = parent;
        }
        else
        {
            break;
        }
    }
}

/* Push the element at 'index' down until heap property holds */
static void heapifyDown(MaxHeap *h, int index)
{
    while (1)
    {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < h->size && emailIsHigherPriority(&h->data[left], &h->data[largest]))
        {
            largest = left;

```

```

    }

    if (right < h->size && emailIsHigherPriority(&h->data[right],
&h->data[largest]))
    {
        largest = right;
    }

    if (largest == index)
    {
        break;
    }

    heapSwap(&h->data[index], &h->data[largest]);
    index = largest;
}
}

static void heapInsert(MaxHeap *h, Email e)
{
    heapEnsureCapacity(h);
    h->data[h->size] = e;
    heapifyUp(h, h->size);
    h->size++;
}

/* Removes and returns the highest priority email.
Returns 1 on success, 0 if the heap was empty. */
static int heapExtractMax(MaxHeap *h, Email *out)
{
    if (h->size == 0)
    {
        return 0;
    }

    if (out != NULL)
    {
        *out = h->data[0];
    }

    h->size--;
    h->data[0] = h->data[h->size];
    heapifyDown(h, 0);
    return 1;
}

/* Returns the highest priority email WITHOUT removing it.

```

```

    Returns 1 on success, 0 if the heap is empty. */
static int heapPeekMax(const MaxHeap *h, Email *out)
{
    if (h->size == 0)
    {
        return 0;
    }
    if (out != NULL)
    {
        *out = h->data[0];
    }
    return 1;
}

/* =====
   Parsing / utility helpers
   ===== */

/* Maps a sender category string to a numeric rank.
   Higher number = higher priority (read sooner). */
static int categoryRank(const char *category)
{
    if (strcmp(category, "Boss") == 0)
        return 5;
    if (strcmp(category, "Subordinate") == 0)
        return 4;
    if (strcmp(category, "Peer") == 0)
        return 3;
    if (strcmp(category, "ImportantPerson") == 0)
        return 2;
    if (strcmp(category, "OtherPerson") == 0)
        return 1;
    return 0; /* unknown category - lowest priority, shouldn't happen */
}

/* Converts "MM-DD-YYYY" into an integer YYYYMMDD so that later dates
   compare as larger numbers. */
static long dateToValue(const char *date)
{
    int month = 0, day = 0, year = 0;
    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3)
    {

```

```

        return 0;
    }
    return (long)year * 10000L + (long)month * 100L + (long)day;
}

/* Strips trailing \n and \r characters from a line read via fgets */
static void trimNewline(char *s)
{
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == '\n' || s[len - 1] == '\r'))
    {
        s[len - 1] = '\0';
        len--;
    }
}

static void printEmail(const Email *e)
{
    printf("Sender: %s\n", e->category);
    printf("Subject: %s\n", e->subject);
    printf("Date: %s\n", e->date);
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness: Both programs produced the same correct output when I tested them with the assignment's sample test file. Both correctly implement the 5 sender categories in the required priority order and use the newest date as the tiebreaker when two emails have the same sender category. Both also safely handle READ when there are no emails. For NEXT when the queue is empty, Claude's version prints "No emails to read.", while Gemini's version does not print anything. The assignment does not specify what the output should be for an empty NEXT, so both approaches are safe. One difference is that Gemini uses strcpy() when copying the sender, subject, and date. This can cause a buffer overflow if the input is longer than the arrays allow. Claude's version uses strncpy() with size limits, which makes the string copying safer.

Execution Time: Both programs have the same Big-O execution time for the main heap operations. Inserting an email and removing the highest priority email are both $O(\log n)$, while NEXT and COUNT are $O(1)$. However, Claude's version does less repeated work during heap operations. Gemini's compare_emails() calls get_sender_rank() and get_date_value() every time two emails are compared. This means it repeatedly uses string comparisons and sscanf() during heapify_up and heapify_down. Claude calculates the category rank and date value when the email is inserted and stores those values in

the Email structure. The heap comparisons can then compare integers instead of repeatedly parsing the strings. This does not change the Big-O complexity, but it reduces the amount of work done during each comparison.

Space Complexity: Both programs have $O(n)$ space complexity because they store the emails in a dynamically allocated array. Both also increase the array capacity by doubling it when more space is needed. Claude's Email structure is slightly larger because it stores categoryRank and dateValue in addition to the original email information. This uses a small amount of extra memory for each email, but it avoids repeatedly calculating those values during heap operations. I think this is a reasonable space-for-time tradeoff.

Maintainability: Both programs are divided into separate functions instead of putting all of the code inside main(), which makes them easier to understand and modify. Claude's version uses a consistent naming style for its heap functions, such as heapInit, heapInsert, heapExtractMax, and heapPeekMax. It also separates the heap operations from the parsing and printing functions. This makes it easier to find and modify specific parts of the program. Another difference is that Claude's heapPeekMax() copies the email into an output variable instead of returning a pointer directly to the heap's internal array. Gemini's peek() returns a pointer to the internal array. This is not a problem for the current program, but returning a pointer to internal data could create problems if the array is later resized with realloc() and the pointer is still being used.

f. Select one to use as the basis for this assignment and justify why you picked it.

I would choose Claude's version as the basis for this assignment. First, Claude's version has safer string handling. Gemini uses strcpy() without checking the size of the destination arrays, while Claude limits the number of characters copied into each array. Second, Claude's version avoids repeatedly calculating the sender priority and date value during heap comparisons. It calculates these values when the email is inserted and stores them in the Email structure. This makes the comparisons during heapifyUp and heapifyDown simpler and reduces repeated work. Both programs correctly implement the required MaxHeap and have the same Big-O complexity, so I chose Claude's version because of its safer memory handling and reduced repeated work.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: I improved the code by using strncpy() with explicit size limits instead of strcpy() when copying the sender, subject, and date. This makes the program safer when handling strings. I also made sure that the program checks whether the heap is empty before trying to remove or display an email. I also added warning messages, printed to stderr instead of the normal output, whenever an EMAIL line is malformed or uses an unrecognized sender category or an unparsable date. This way bad input doesn't cause a crash or get silently misprioritized, and it doesn't affect the graded terminal output.

Claude's code already handles the required priority rules correctly, so I kept those rules the same.

Execution Time: I kept the MaxHeap operations because they already have the correct $O(\log n)$ time for inserting and removing emails and $O(1)$ time for NEXT and COUNT. I improved the comparisons by calculating the sender category rank and date value once when the email is inserted instead of recalculating them every time two emails are compared. I also changed the heapInsert function to take the email as a pointer instead of copying the whole struct by value, which avoids an unnecessary extra copy of all the email's data every time a new email is added.

Space Complexity: The space complexity remains $O(n)$ because the program still uses one dynamically allocated array to store the emails. I did not add another data structure that would store additional copies of all the emails. The extra categoryRank and dateValue fields use a small amount of additional memory, but this is a deliberate tradeoff to reduce repeated calculations.

Maintainability: I kept the heap operations, input parsing, and printing in separate functions. I also kept a consistent naming convention for the heap functions and added comments explaining the priority rules and the main heap operations. I renamed the helper functions categoryRank() and dateToValue() to getCategoryRank() and getDateValue() because their original names were too similar to the struct fields they filled in (categoryRank and dateValue), which could confuse someone reading the code later. I also replaced the hardcoded starting heap size (8) with a named constant, INITIAL_HEAP_CAPACITY, so the value's purpose is clear and easy to change in one place. This makes the code easier to read and makes it easier to modify one part of the program without changing the entire program.